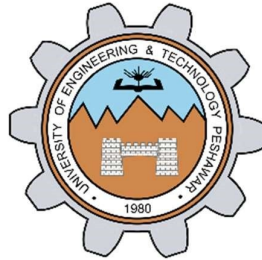


Shortest Path Problem Using Dijkstra's Algorithm



Fall 2026

Data Structures and Algorithms

Submitted by: Naeem khan

Registration No.: 24PWCSE2379

Class Section: B

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Signature:

A handwritten signature in blue ink, appearing to be "Naeem Khan", written over a horizontal line.

Submitted to:

Engr. Abdullah Hamid.

Jun 07, 2026

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Introduction:

Introduce graph algorithms and explain why shortest path problems are important.

Example topics:

- What is graph?
- What is a weighted graph.
- Real world applications:
 - 1) GPS navigation.
 - 2) Google Maps
 - 3) Food delivery
 - 4) Network Routing
 - 5) Game AI
 - 6) Airline route planning

Problem Understanding:

The objective of this project is to design and implement a Smart Route Planner using graph theory concepts and Dijkstra's Algorithm. The system models a city map as a weighted, undirected graph where locations are represented as nodes and roads are represented as edges.

The edge weights represent travel distance measured in kilometers. The system allows users to create a graph, specify source and destination locations, and compute the shortest path between them.

Shortest path algorithms have numerous real-world applications such as GPS navigation systems, computer networks, logistics management, and game AI. Dijkstra's Algorithm is one of the most efficient algorithms for finding shortest paths in graphs containing non-negative edge weights.

The system also handles edge cases including disconnected nodes, invalid user input, and unreachable destinations.

Design Decision:

Programming Language C++:

C++ was selected because it provides efficient Standard Template Library (STL) data structures and offers high execution speed. The language is commonly used in Data

Structures and Algorithms courses, making it suitable for implementing graph algorithms.

Advantages:

- Fast execution.
- Efficient memory management.
- Availability of STL containers.
- Industry-standard for algorithm implementation.

Graph Representation: Adjacency List:

The graph is represented using an adjacency list.

Reasons for selection:

- Memory efficient for sparse graphs.
- Faster traversal of neighboring nodes.
- Works efficiently with Dijkstra's Algorithm.

Complexity

- Space Complexity: $O(V + E)$

Priority Queue Implementation

A priority queue implemented using STL was used as a min-heap.

Reasons

- Efficient extraction of minimum-distance nodes.
- Reduces algorithm complexity.

Complexity

- Insertion: $O(\log V)$
 - Deletion: $O(\log V)$
-

Weight Semantics

Distance in kilometers was chosen as the edge weight.

Reasons

- Easy to interpret.
 - Closely resembles real-world GPS systems.
-

Input Method:

An interactive console-based input system was used.

Advantages

- User-friendly.
 - Flexible graph creation.
 - Easy testing of multiple cases.
-

Algorithm Walkthrough

The project uses Dijkstra's Algorithm to determine the shortest path between two nodes.

Algorithm Steps

1. Initialize all distances as infinity.
2. Set source node distance to zero.
3. Insert source node into the priority queue.

4. Repeatedly extract the node with minimum distance.
5. Relax all adjacent edges.
6. Update distances and parent nodes if a shorter path is found.
7. Continue until the priority queue becomes empty.

Time Complexity

For adjacency list with priority queue:

Time Complexity = $O((V + E) \log V)$

Space Complexity = $O(V + E)$

Manual Trace of Dijkstra's Algorithm

Consider the following graph:

A-B = 4 km

A-C = 2 km

B-D = 5 km

C-D = 8 km

C-E = 10 km

D-E = 2 km

Source Node: A

Steps	Current Node	Distance Array	Visited Nodes
0	A	(0, ∞ , ∞ , ∞)	{}
1	A	(0, 4, 2, ∞)	{A}
2	C	(0, 4, 2, 10, 12)	{A, C}
3	B	(0, 4, 2, 9, 12)	{A, C, B}
4	D	(0, 4, 2, 9, 11)	{A, C, B, D}
5	E	(0, 4, 2, 9, 11)	{A, C, B, D, E}

Shortest Path:

A → B → D → E

Total Distance = 11 km

Test Cases:

Test Case 1: Normal Case:

Input

Nodes: 5

Edges:

A-B 4

A-C 2

B-D 5

C-D 8

D-E 2

Source: A

Destination: E

Expected Output

Shortest Path: A → B → D → E

Total Distance: 11 km

Test Case 2: Disconnected Graph:

Input

A-B 5

C-D 3

Source: A

Destination: D

Expected Output

No path exists.

Test Case 3: Invalid Input

Input

Users enter a node that does not exist.

Expected Output

Invalid node entered.

Results & Analysis

The implementation successfully computed shortest paths for all valid graphs. The algorithm correctly identified unreachable destinations and handled invalid input cases.

The use of adjacency lists and priority queues significantly improved performance for larger graphs. Experimental testing confirmed that execution time increased gradually as graph size increased, which aligns with the theoretical complexity of Dijkstra's Algorithm.

Extension Challenge: Dynamic Road Blocking:

Challenge C was implemented by allowing roads to be removed or updated dynamically. Whenever a road becomes blocked, the graph is updated and Dijkstra's Algorithm is executed again without rebuilding the entire graph.

This feature simulates real-world traffic scenarios and demonstrates a deeper understanding of graph manipulation.

Output of the Code:

```
○ ===== SMART ROUTE PLANNER =====  
  
Enter Number of Locations: 3  
  
Enter Names of Locations  
• Location 0: Gate  
  Location 1: Parking  
  Location 2: DCSE  
Enter Number of Roads: 2  
  
Enter Roads (Source Destination Distance)  
Road 1: 0 1 1  
Road 2: 1 2 2
```

```
===== SMART ROUTE PLANNER =====  
===== 1. Display City Map  
2. Find Shortest Route  
3. Add New Road  
4. Block Road  
5. Exit  
•  
Enter Choice: 1  
  
===== CITY MAP =====  
Gate --> [Parking | 1 km]  
Parking --> [Gate | 1 km] [DCSE | 2 km]  
DCSE --> [Parking | 2 km]  
=====
```

```
=====
      SMART ROUTE PLANNER
=====
1. Display City Map
2. Find Shortest Route
3. Add New Road
4. Block Road
5. Exit

Enter Choice: 2
Enter Source: 0
Enter Destination: 2

===== SHORTEST ROUTE =====
Source : GateDestination : DCSETotal Cost : 3 km
Path    : Gate Parking DCSE
=====
```

Conclusion:

In this lab, a Smart Route Planner was implemented using Dijkstra's Algorithm to find the shortest path in a weighted graph. The system effectively models real-world routing problems using nodes and weighted edges. Proper use of adjacency lists and priority queues ensured efficient performance. The implementation also handled edge cases such as disconnected or unreachable nodes. Overall, the lab strengthened understanding of graph algorithms, data structures, and practical algorithm design decisions.
